UNSUPERVISED MACHINE LEARNING

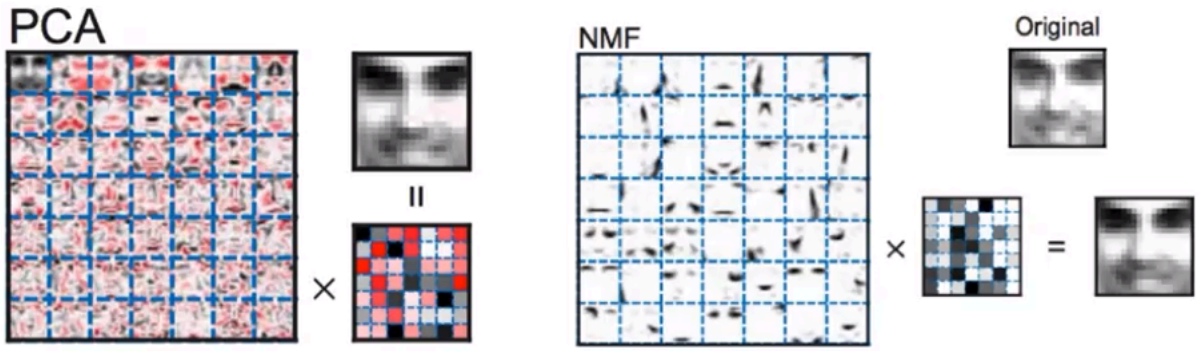
MODULE 6:

MATRIX FACTORIZATION

TABLE OF CONTENTS

Introduction: Non-negative Matrix Factorization.....2
Kernel Principal Component Analysis (Kernel PCA).....2
Why Use NMF?.....3
Applications of NMF..... 3
Python Implementation.....4
Comparison with PCA and Other Methods.....4
Key Takeaways.....5

Introduction: Non-negative Matrix Factorization



Concept:

- Non-Negative Matrix Factorization (NMF) is another **dimensionality reduction** method, similar to PCA.
- It decomposes a data matrix V into two smaller matrices W and H, such that:

$$V \approx W \times H$$

- However, unlike PCA, **NMF requires all values to be non-negative**, which makes it suitable for data like:
  - Image pixels (non-negative intensity values)
  - Word frequency or TF-IDF matrices in text data
- Because NMF only adds positive components, it often produces **more interpretable results**.

Kernel Principal Component Analysis (Kernel PCA)

Matrix Decomposition Form:

$$V_{(m\times n)} \approx W_{(m\times k)} \times H_{(k\times n)}$$

- V: Original data (e.g., documents × words or images × pixels)
- W: Basis matrix — describes how terms or features contribute to topics
- H: Coefficient matrix — describes how topics combine to reconstruct the data

Constraint:
All entries in V, W, and H must be **non-negative**.

Why Use NMF?

Advantages:

- Produces **interpretable features** — each component adds meaningfully to the whole.
  Example: In facial recognition, separate components may represent *nose*, *eyes*, or *mouth*.
- Ensures **additive combinations only**, avoiding cancellation from negative values.

Disadvantages:

- **Information loss** due to truncation of negative values to zero.
- Components are **not orthogonal**, meaning vectors can point in similar directions.
- Sensitive to **initialization** and may produce different results for different random seeds.

Applications of NMF

- **Text Mining & Topic Modeling**
  → Identify topics from document-word matrices.

- **Image Processing**
  → Extract interpretable visual features.
- **Speech/Audio Processing**
  → Separate sources in music or video signals.
- **Data Compression**
  → Encode large non-negative data efficiently.

Python Implementation

```
from sklearn.decomposition import NMF

# Create an instance of the NMF model
nmf = NMF(n_components=3, init='random')

# Fit and transform the data
X_nmf = nmf.fit_transform(X)
```

Optionally, inspect components:

```
nmf.components_
```

Comparison with PCA and Other Methods

Method	Handle Negatives?	Preserves Variance?	Interpretable Components?	Nonlinear?
PCA	Yes	Yes	Hard to interpret	Linear
Kernel PCA	Yes	Yes	No	Nonlinear
MDS	Yes	No	Visualization	Nonlinear
NMF	Only Positive	Not based on variance	Highly interpretable	Nonlinear additive

Key Takeaways

- **NMF** factorizes data into two positive matrices $\boldsymbol{W}$ and $\boldsymbol{H}$, where $\boldsymbol{V} \approx \boldsymbol{WH}$
- Each component in $\boldsymbol{W}$ and $\boldsymbol{H}$ represents a **meaningful, additive part** of the data.
- Ideal for applications requiring **interpretability** (e.g., topics, image parts).
- Sensitive to random initialization — results may vary.
- Best suited for **non-negative, sparse, or count-based data**.